



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Adelia
NPM	2428240104
Kelas	SI5B
Topik Tugas 1	Studio Foto – Paket Foto
Entitas utama	photoPackage
Tautan repositori	adelia2428240104-coder/si5b_kuis1_Adelia
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	08 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Studio Foto – Paket Foto
Entitas utama (tunggal)	photoPackage
Alamat dasar	/photo-packages
Field data	Id, namaPaket, jenis, jumlahFoto, durasiJam, harga
Field wajib saat POST	namaPaket, jenis, jumlahFoto, harga
Query string untuk filter	jenis
Data awal	3 data paket foto
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/photo-packages	Tidak	200	Ambil semua data paket foto, termasuk filter
2	GET	/photo-packages/:id	Tidak	200	Ambil satu paket foto berdasarkan ID
3	POST	/photo-packages	Ya	201	Tambah data paket foto
4	PUT	/photo-packages/:id	Ya	200	Ubah data paket foto
5	DELETE	/photo-packages/:id	Ya	204	Hapus data paket foto
6	GET	/photo-packages?jenis=wisuda	Tidak2	200	Filter paket foto

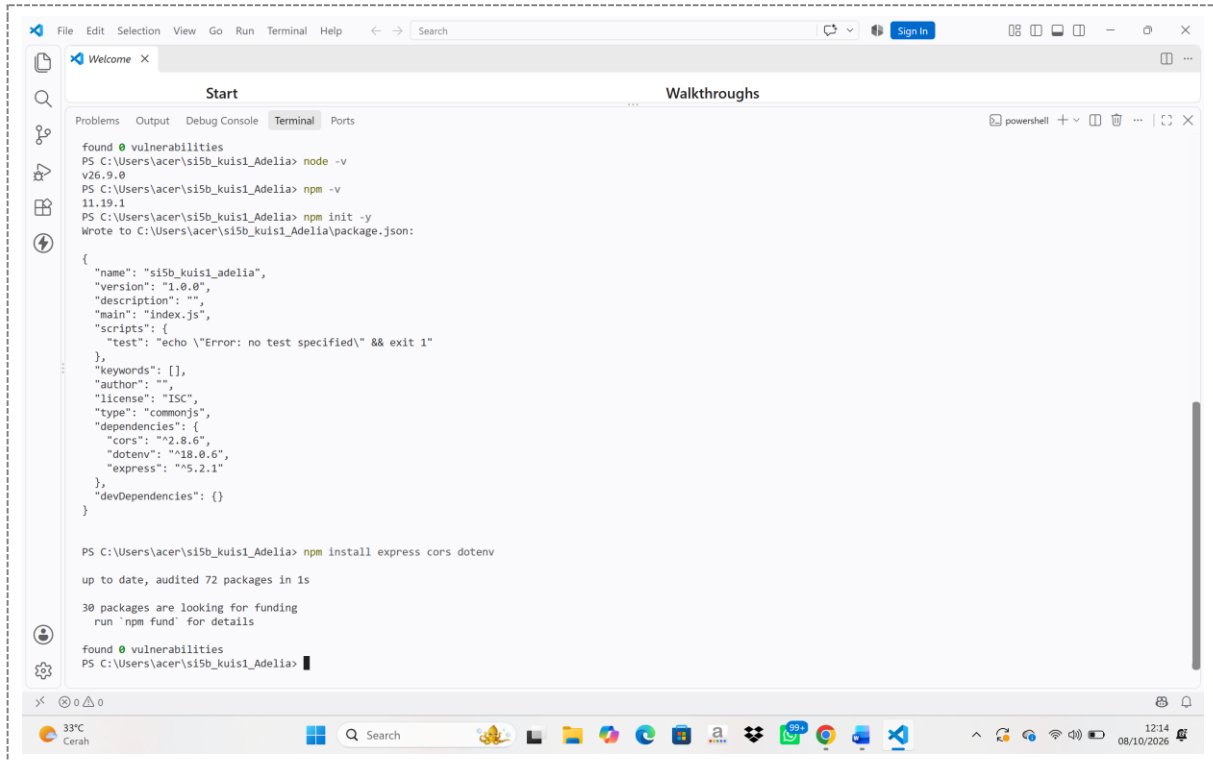
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```

PS C:\Users\acer\si5b_kuis1_Adelia> node -v
v26.9.0
PS C:\Users\acer\si5b_kuis1_Adelia> npm -v
11.19.1
PS C:\Users\acer\si5b_kuis1_Adelia> npm init -y
Wrote to C:\Users\acer\si5b_kuis1_Adelia\package.json:

{
  "name": "si5b_kuis1_adelia",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^18.0.6",
    "express": "^5.2.1"
  },
  "devDependencies": {}
}

PS C:\Users\acer\si5b_kuis1_Adelia> npm install express cors dotenv
up to date, audited 72 packages in 1s
30 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
PS C:\Users\acer\si5b_kuis1_Adelia>
  
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Express.js digunakan untuk membantu membuat RESTful API pada project karena menyediakan fitur untuk membuat route dan mengatur request serta response dari client. CORS digunakan untuk mengatur izin akses dari sumber atau domain yang berbeda ketika API digunakan. Sedangkan dotenv digunakan untuk membaca konfigurasi yang disimpan dalam environment variable sehingga data konfigurasi tidak perlu ditulis langsung di dalam kode program.

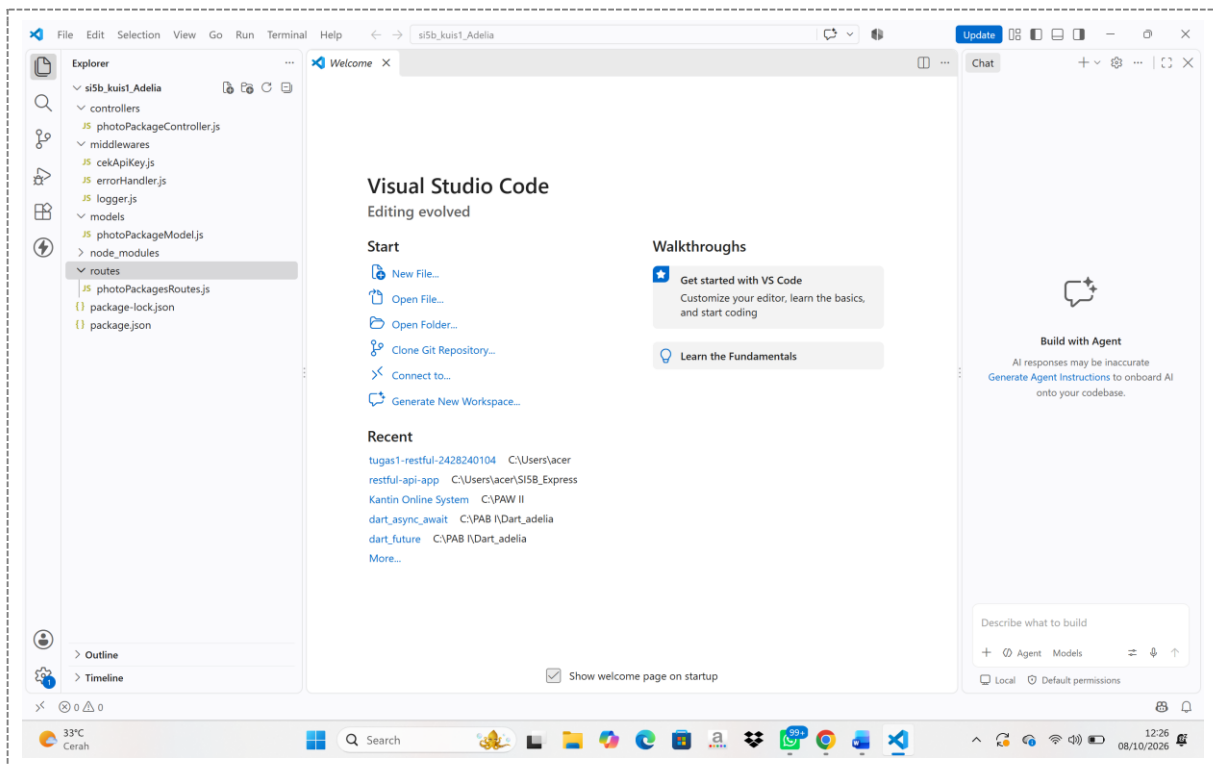
Versi Node.js juga perlu diperhatikan karena Express 5 membutuhkan versi Node.js yang mendukung fitur-fitur yang digunakan oleh Express. Pada project ini digunakan Node.js versi 26.9.0 sehingga sudah memenuhi kebutuhan untuk menjalankan Express 5.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

- **Controllers** : Menangani request, validasi data, dan response dari API.
- **Models** : Menyimpan data paket foto serta fungsi untuk mengolah data tersebut.
- **Routes** : Mengatur endpoint API dan menghubungkannya dengan fungsi pada controller.
- **Middlewares** : Menangani proses tambahan seperti logger, pengecekan API key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

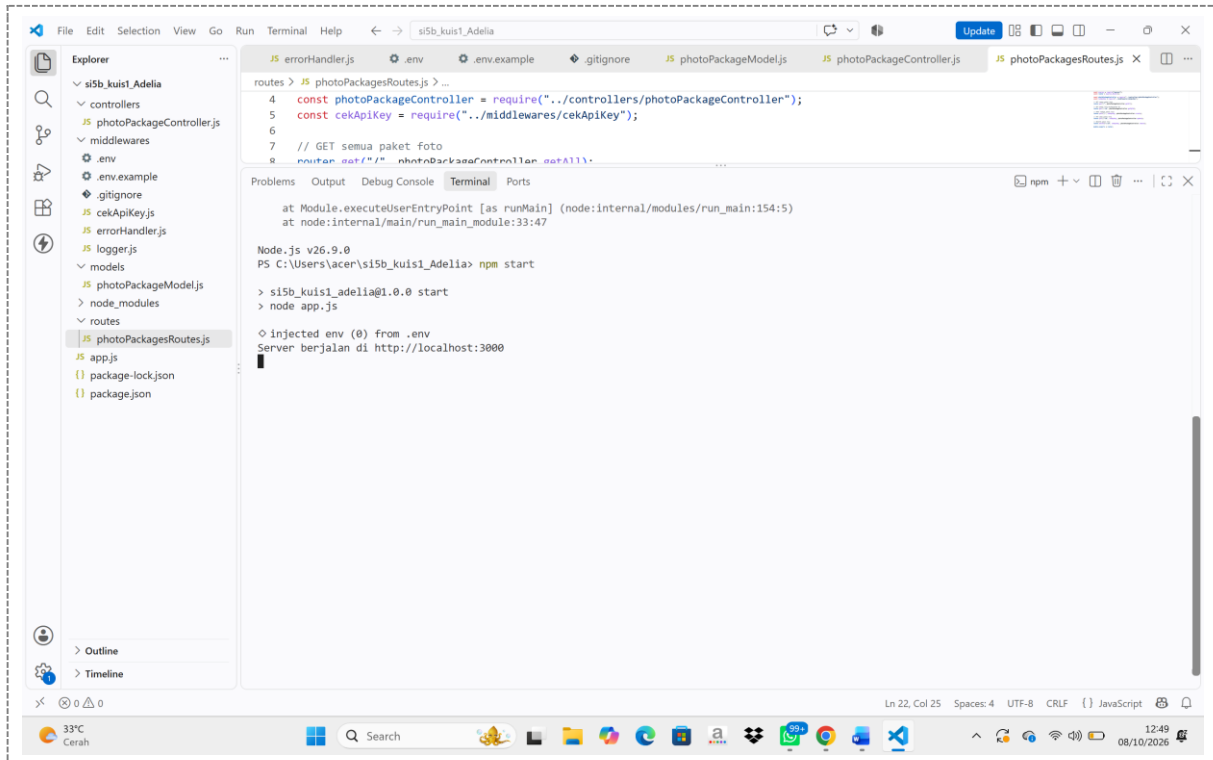
Berkas	Penjelasan Anda
models/photopackageModel.js	Berisi data photoPackages dan fungsi untuk mengambil, menambah, mengubah, serta menghapus data paket foto.
controllers/photoPackageController.js	Berisi fungsi controller untuk menerima request, melakukan validasi, memanggil model, dan mengirim response API.

Berkas	Penjelasan Anda
routes/photoPackageRoutes.js	Mengatur endpoint /photo-packages dan menghubungkannya dengan fungsi yang ada di controller menggunakan express.Router().
middlewares/logger.js	Berfungsi mencatat informasi request yang masuk seperti method dan URL yang diakses.
middlewares/cekApiKey.js	Berfungsi mengecek API key sebelum request POST, PUT, dan DELETE diproses.
middlewares/errorHandler.js	Berfungsi menangani error secara terpusat dan memberikan response JSON ketika terjadi kesalahan, termasuk route yang tidak ditemukan.
app.js	Menjadi file utama untuk menjalankan Express, memasang middleware, menghubungkan routes, dan menjalankan server.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port server berasal dari pengaturan PORT yang disimpan pada file `.env`, yaitu `PORT=3000`. Pengaturan tersebut kemudian digunakan pada `app.js` untuk menentukan port yang digunakan oleh server Express. Jika port 3000 sudah digunakan oleh aplikasi lain, server tidak dapat dijalankan pada port tersebut dan akan menampilkan error karena satu port tidak dapat digunakan oleh dua aplikasi secara bersamaan. Solusinya adalah menghentikan aplikasi yang menggunakan port tersebut atau mengganti nomor port pada file `.env`.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

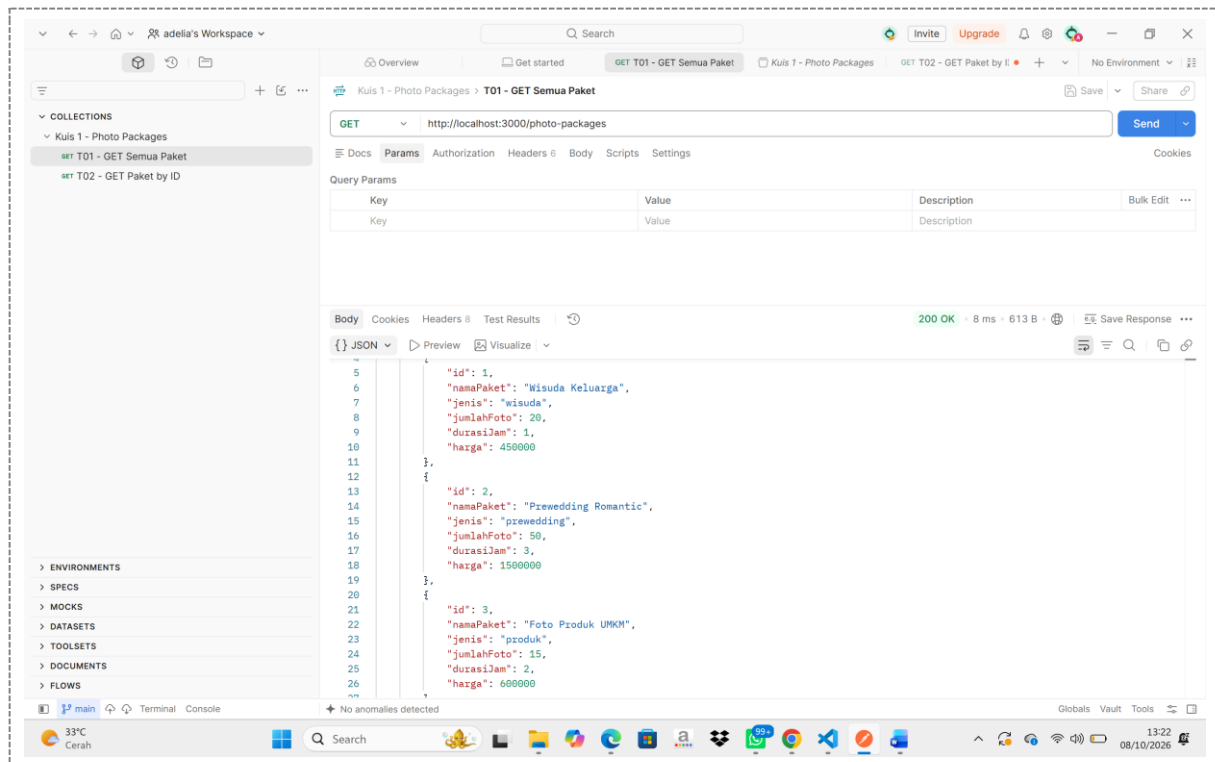
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /photo-packages
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; data seluruh paket foto berhasil ditampilkan.
Lulus?	[☒] Ya [☐] Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

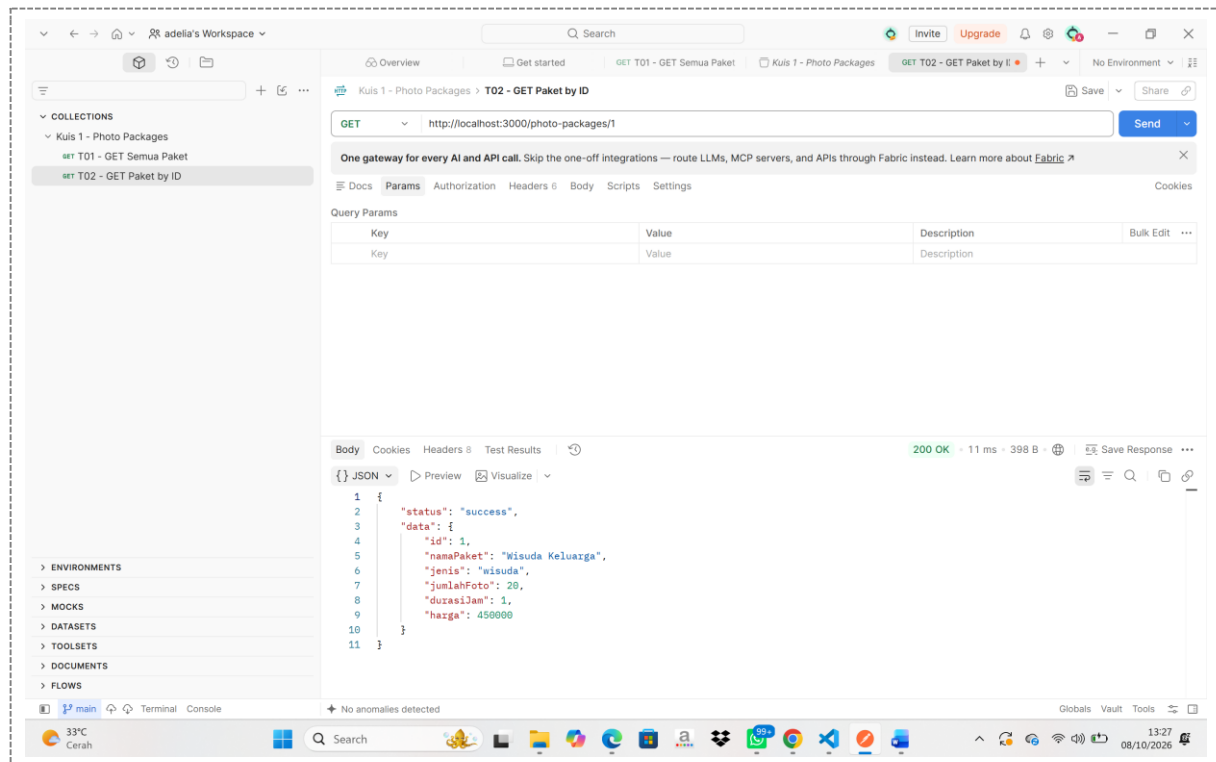
Kode status yang muncul adalah 200 OK, yang berarti request berhasil diproses oleh server. Endpoint ini digunakan untuk mengambil dan menampilkan seluruh data paket foto yang tersedia pada sistem. Response menampilkan tiga data paket foto yang terdapat pada Model.

Urutan pemanggilan dimulai dari Route GET `/photo-packages`, kemudian diteruskan ke fungsi **Controller** `getAll`. Controller memeriksa apakah terdapat parameter `jenis` pada request. Karena pada pengujian ini tidak ada parameter tersebut, Controller memanggil fungsi Model `getAllPhotoPackages()` untuk mengambil seluruh data paket foto. Data tersebut kemudian dikirim kembali kepada client dalam bentuk response JSON dengan status 200.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /photo-packages/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; data paket foto dengan ID 1 berhasil ditampilkan.
Status hasil uji	200 OK; data paket foto dengan ID 1 berhasil ditampilkan.
Lulus?	[✓] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

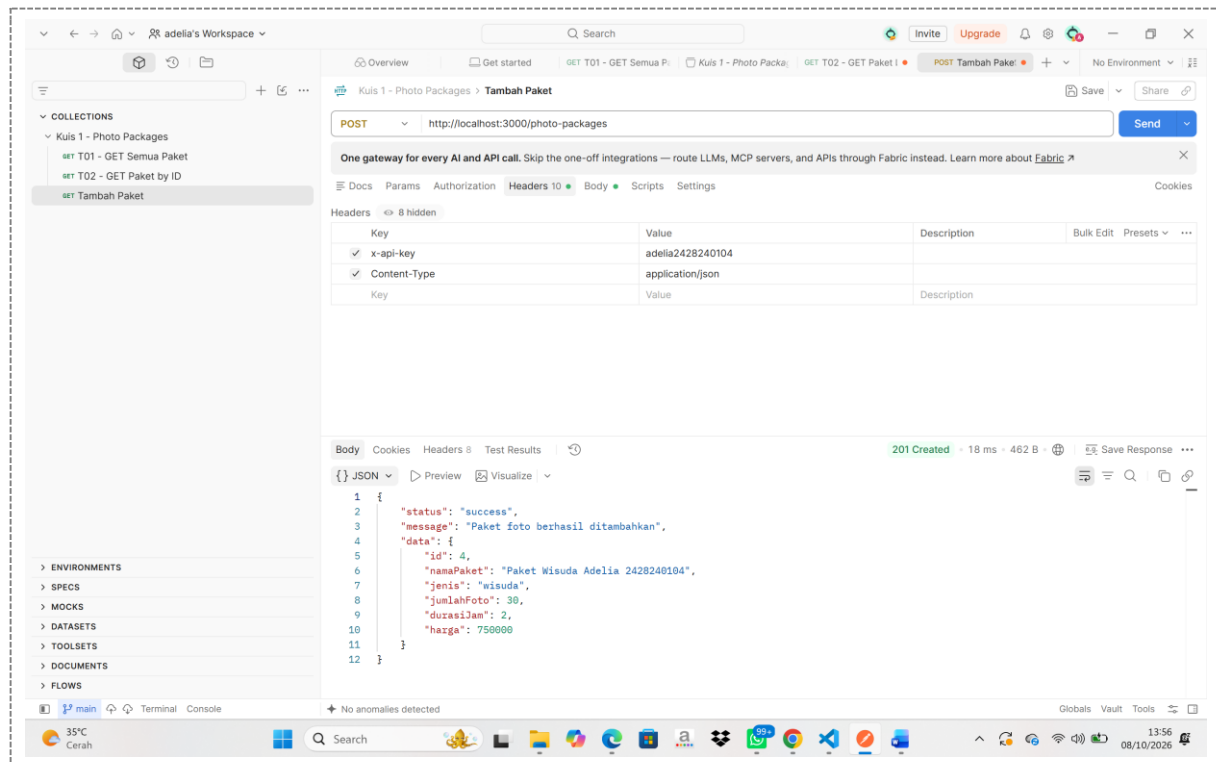
Penjelasan:

ID diambil dari alamat URL menggunakan req.params.id, karena ID terdapat pada bagian akhir alamat /photo-packages/1. Nilai yang diperoleh dari req.params berupa string, sehingga perlu diubah menjadi angka menggunakan parseInt() agar dapat dibandingkan dengan ID yang tersimpan pada Model yang bertipe angka. Setelah diubah menjadi angka, ID tersebut digunakan oleh Controller untuk memanggil fungsi getPhotoPackageById(id) pada Model.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /photo-packages
Header	x-api-key: adelia2428240104 Content-Type: application/json
Body	{"namaPaket": "Paket Wisuda Adelia 2428240104", "jenis": "wisuda", "jumlahFoto": 30, "durasiJam": 2, "harga": 750000}
Hasil yang diharapkan	201 Created; data paket foto berhasil ditambahkan.
Status hasil uji	201 Created; paket foto berhasil ditambahkan dengan ID 4.
Lulus?	[✓] Ya [] Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

Header x-api-key digunakan sebagai pengaman pada endpoint POST. Header tersebut diperiksa oleh middleware cekApiKey sebelum request diteruskan ke Controller. Jika API key tidak ada atau tidak sesuai dengan API key yang tersimpan pada .env, request akan ditolak dengan status 401.

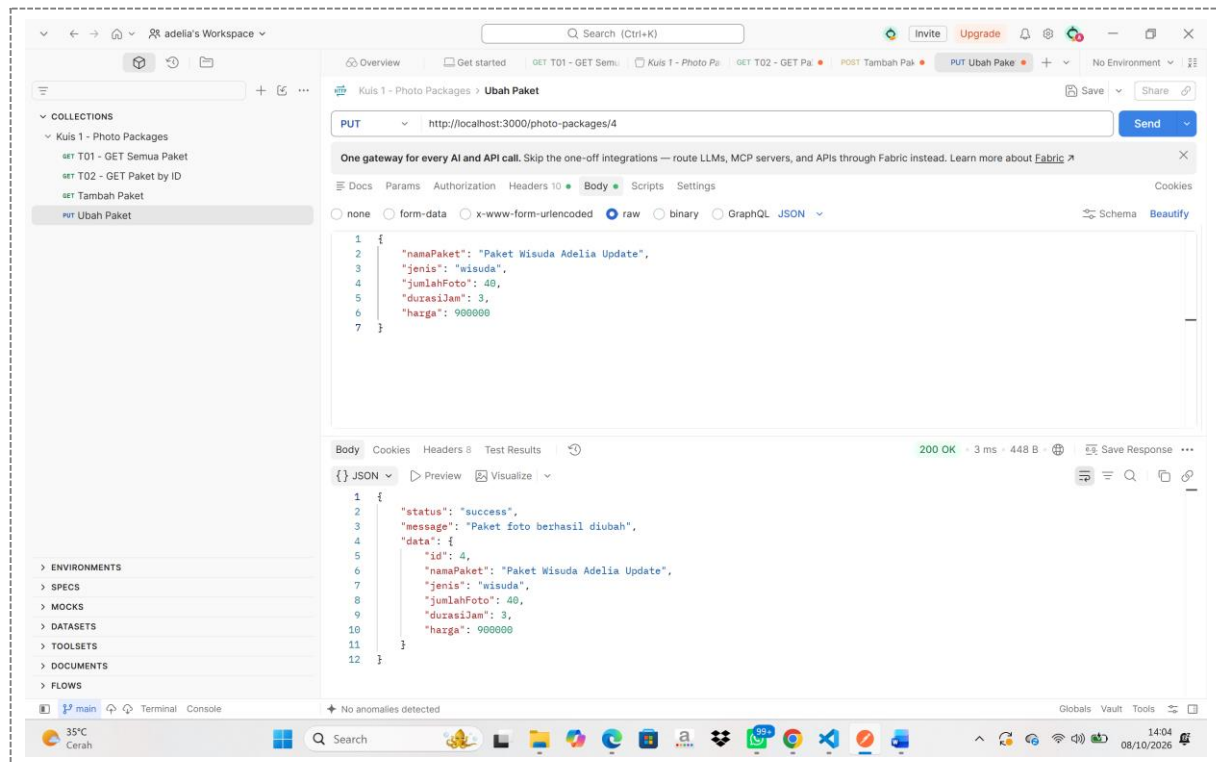
express.json() digunakan agar Express dapat membaca data yang dikirim oleh client dalam format JSON. Dengan middleware ini, data pada request body dapat diakses melalui req.body dan kemudian diproses oleh Controller.

Status yang digunakan adalah **201 Created** karena request POST berhasil membuat atau menambahkan data baru ke dalam sistem. Status 200 lebih umum digunakan ketika request berhasil diproses tanpa menunjukkan bahwa sebuah resource baru telah dibuat.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /photo-packages/4
Header	x-api-key: adelia2428240104 Content-Type: application/json
Body	{ "namaPaket": "Paket Wisuda Adelia Update", "jenis": "wisuda", "jumlahFoto": 40, "durasiJam": 3, "harga": 900000 }
Hasil yang diharapkan	200 OK; data paket foto berhasil diubah.
Status hasil uji	200 OK; data paket foto dengan ID 4 berhasil diubah.
Lulus?	[✓] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

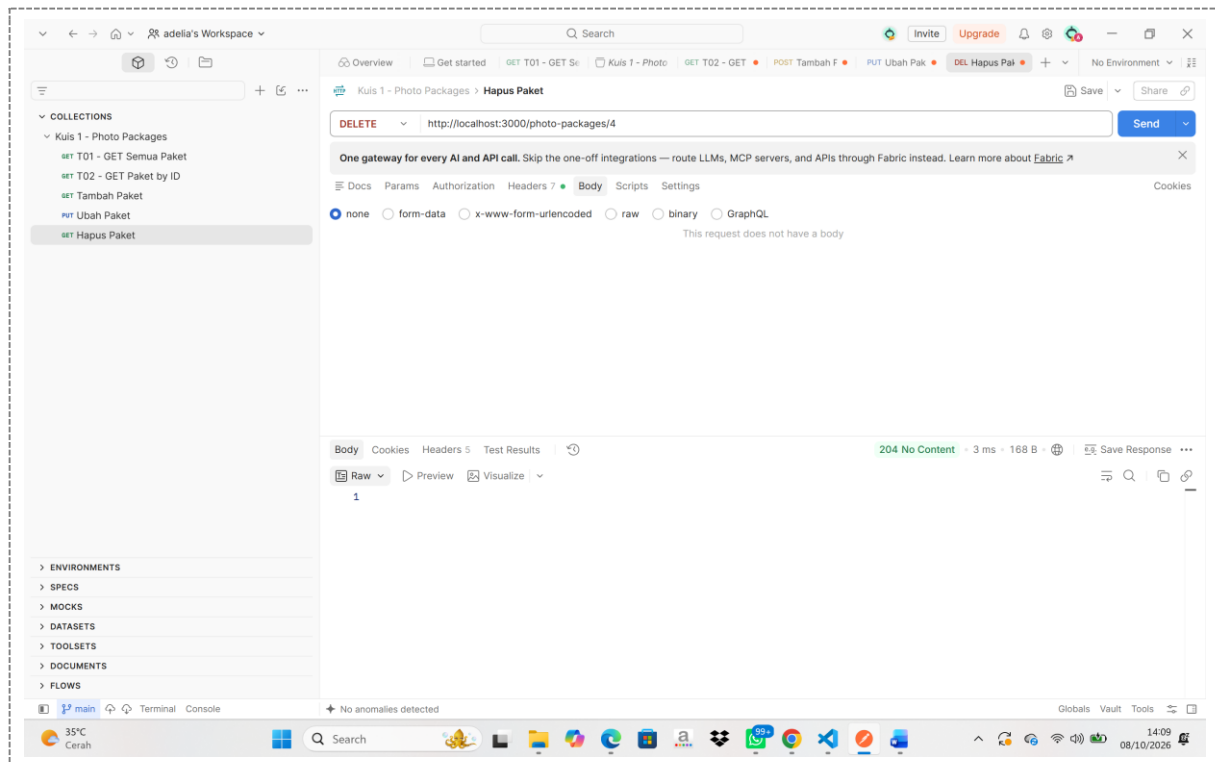
Pada pengujian T04, field yang berubah adalah namaPaket, jumlahFoto, durasiJam, dan harga. Sebelumnya data ID 4 memiliki nama “Paket Wisuda Adelia 2428240104”, jumlah foto 30, durasi 2 jam, dan harga 750000. Setelah diperbarui, nilainya menjadi “Paket Wisuda Adelia Update”, jumlah foto 40, durasi 3 jam, dan harga 900000. Field id dan jenis tetap, yaitu ID 4 dan jenis wisuda.

Perubahan data dilakukan pada layer Model melalui fungsi `updatePhotoPackage(id, data)`. Controller hanya menerima request, melakukan validasi, dan memanggil fungsi tersebut, sedangkan Model yang melakukan perubahan terhadap data yang tersimpan.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content menunjukkan bahwa permintaan DELETE berhasil diproses dan data paket foto dengan ID 4 berhasil dihapus. Respons tidak berisi data apa pun karena setelah proses penghapusan tidak ada informasi yang perlu dikirim kembali kepada client. Oleh karena itu, endpoint DELETE menggunakan status 204 dan tidak mengirimkan response body.

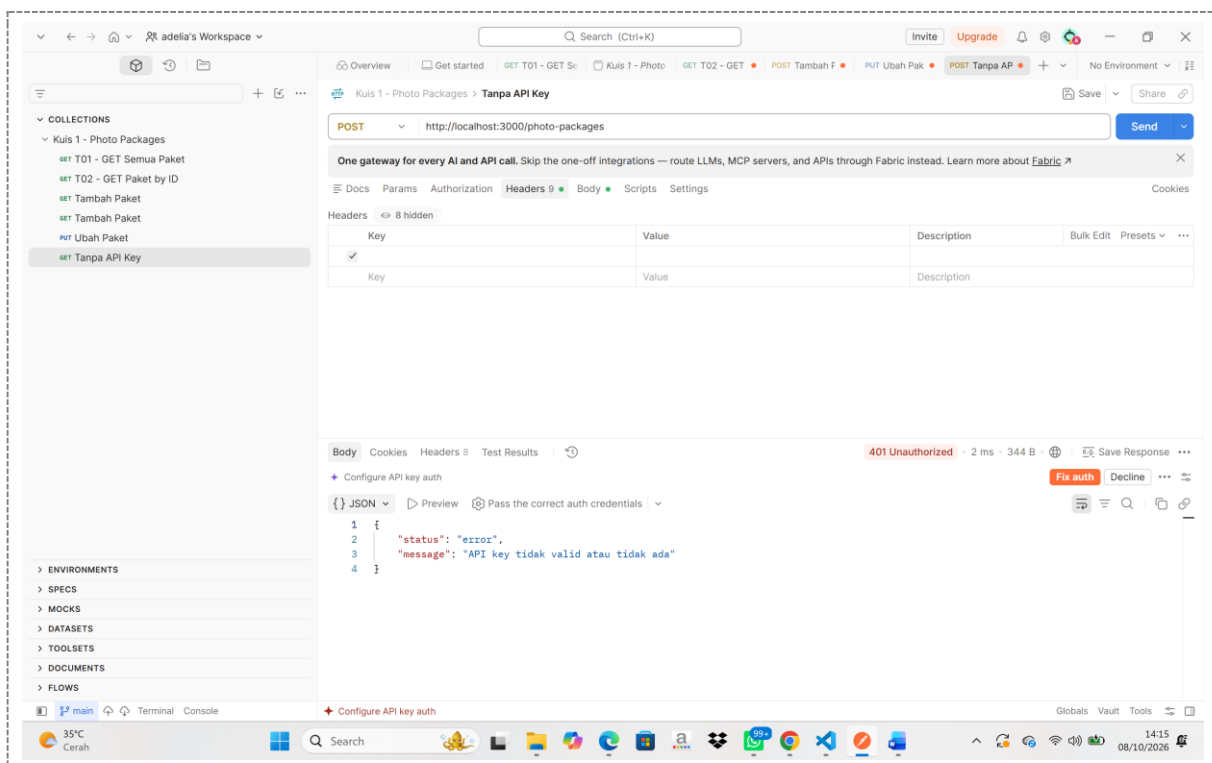
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /photo-packages (tanpa x-api-key)
Header	Content-Type: application/json
Body	Data paket foto yang valid
Hasil yang diharapkan	401 Unauthorized; response berupa JSON dan data tidak berubah
Status hasil uji	401 Unauthorized — API key tidak valid atau tidak ada
Lulus?	[✓] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

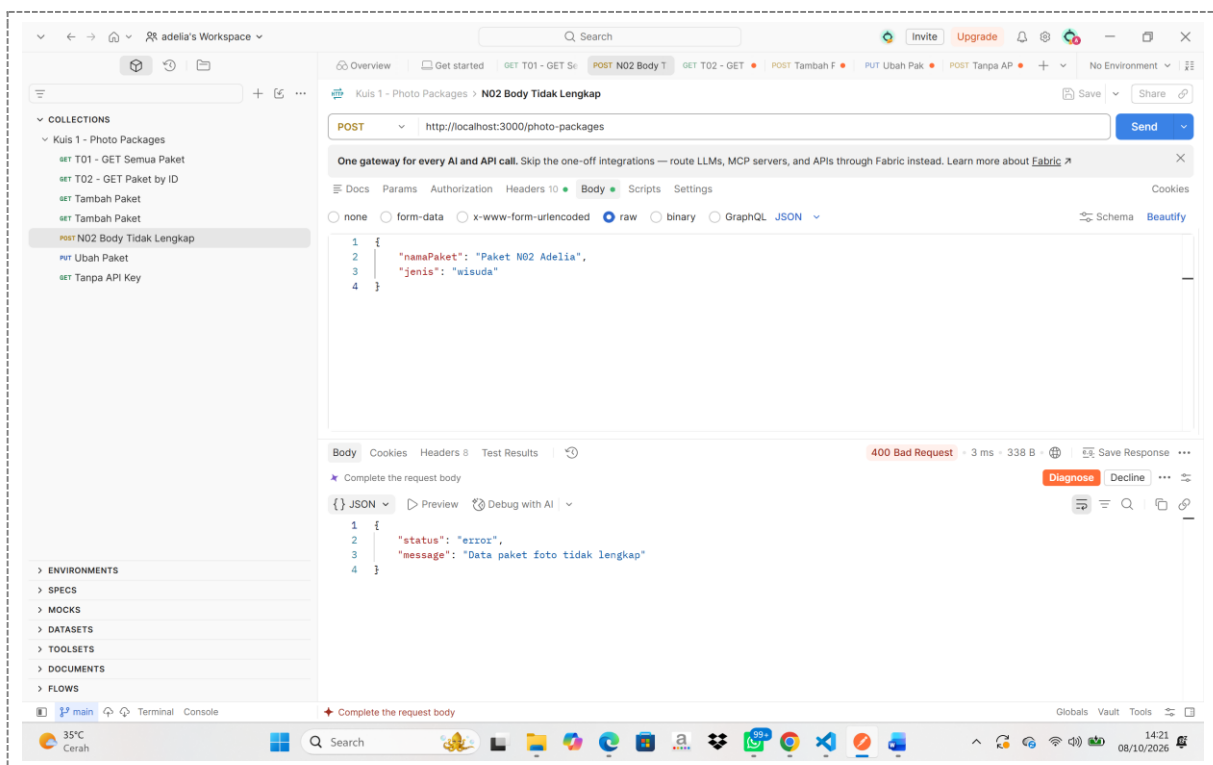
Penjelasan:

Permintaan ditolak oleh middleware cekApiKey karena request POST tidak memiliki API key yang valid pada header x-api-key. Middleware melakukan pengecekan terlebih dahulu sebelum request diteruskan ke Controller. Karena API key tidak ada, middleware langsung mengirim response 401 Unauthorized dan tidak memanggil next(). Akibatnya, request berhenti di middleware dan Controller tidak sempat dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /photo-packages
Header	x-api-key: adelia2428240104 (Content-Type: application/json
Body	namaPaket dan jenis saja; jumlahFoto dan harga tidak diisi
Hasil yang diharapkan	400 Bad Request; pesan menunjukkan data paket foto tidak lengkap
Status hasil uji	400 Bad Request — Data paket foto tidak lengkap
Lulus?	[✓] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

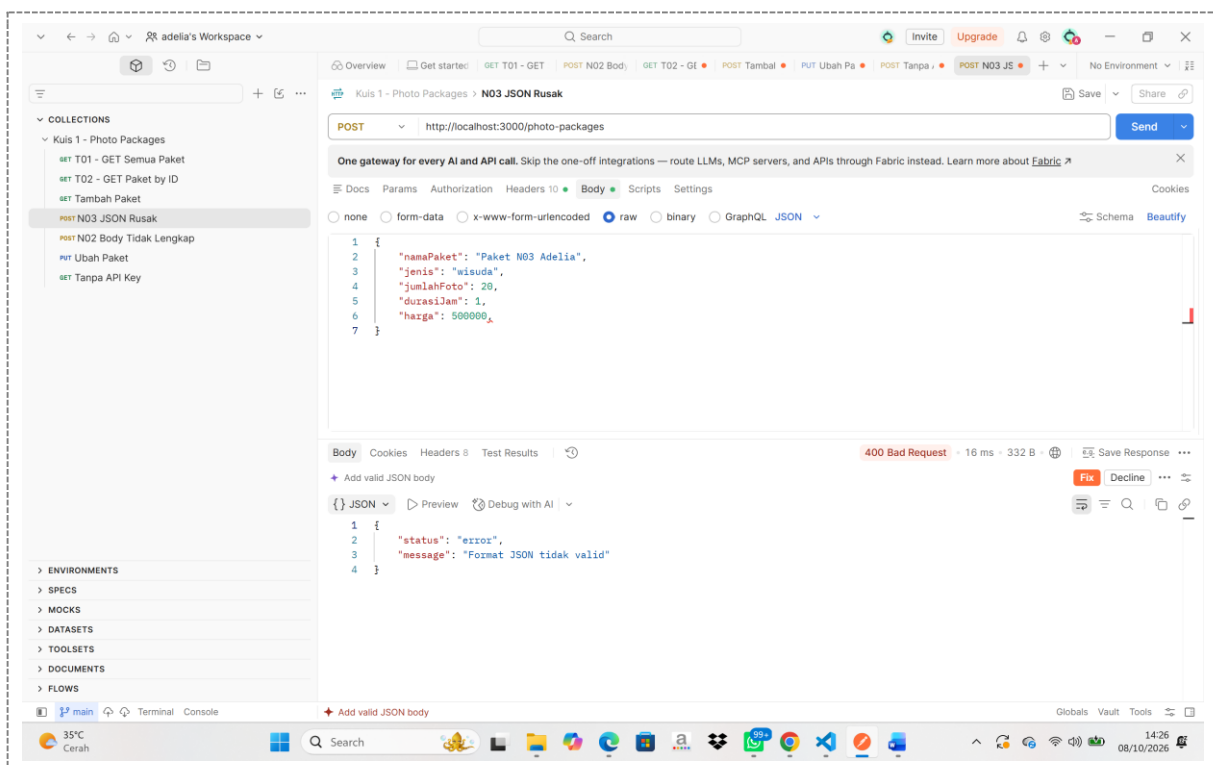
Penjelasan:

Validasi data dilakukan pada layer Controller sebelum data diteruskan ke Model. Pada pengujian ini beberapa field wajib tidak dikirim dalam request body, sehingga Controller mendeteksi bahwa data paket foto belum lengkap dan mengembalikan status 400 Bad Request. Karena validasi gagal, fungsi Model untuk menambahkan data tidak dijalankan sehingga tidak ada data baru yang ditambahkan.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /photo-packages
Header	x-api-key: adelia2428240104 Content-Type: application/json
Body	JSON tidak valid karena terdapat koma berlebih setelah nilai harga
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid"
Status hasil uji	400 Bad Request — Format JSON tidak valid
Lulus?	[✓] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

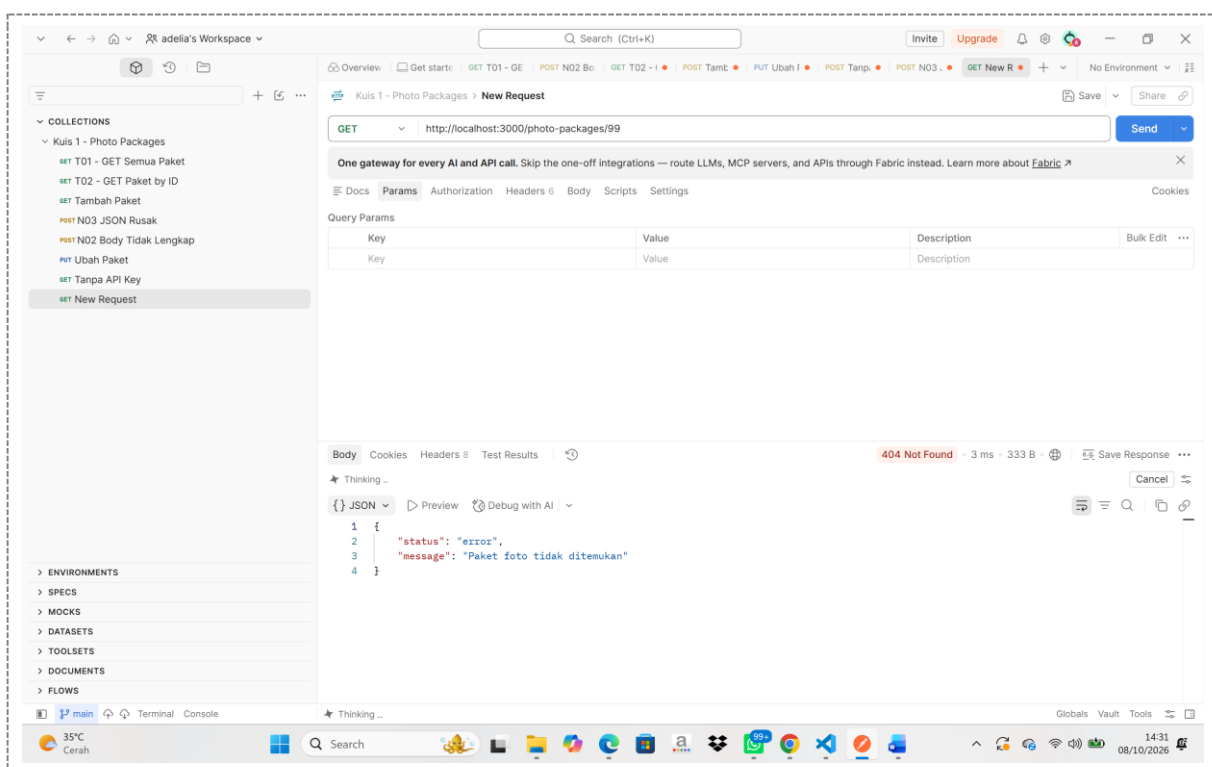
Penjelasan:

JSON yang rusak dideteksi oleh middleware `express.json()` ketika request memiliki format JSON yang tidak valid. Error tersebut kemudian diteruskan ke `errorHandler` sebagai penanganan error terpusat. `errorHandler` mengubah error tersebut menjadi response JSON dengan status 400 dan pesan "Format JSON tidak valid", sehingga API tidak menampilkan halaman HTML atau error mentah.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /photo-packages/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — [isi pesan yang muncul di Postman]
Lulus?	[✓] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request GET dengan ID 99 diteruskan ke route GET /:id, kemudian Controller mengambil nilai ID dari req.params dan mengubahnya menjadi angka. Controller memanggil fungsi Model untuk mencari data dengan ID tersebut. Karena ID 99 tidak ditemukan, Controller membuat error dengan status 404 dan meneruskannya ke errorHandler. Selanjutnya errorHandler mengirimkan response JSON dengan status 404.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	[✓] Ya
T02	Menampilkan satu data berdasarkan ID	200	200	[✓] Ya
T03	Menambah data baru	201	201	[✓] Ya
T04	Mengubah data	200	200	[✓] Ya
T05	Menghapus data	204	204	[✓] Ya
N01	Menambah data tanpa API key	401	401	[✓] Ya
N02	Menambah data dengan body tidak lengkap	400	400	[✓] Ya
N03	Mengirim JSON yang rusak	400	400	[✓] Ya
N04	Mengambil ID yang tidak ada	404	404	[✓] Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/adelia2428240104-coder/si5b_kuis1_Adelia
Pesan commit terakhir	refactor backend into routes controllers models and middlewares
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route berperan menentukan endpoint API dan menghubungkan request ke Controller, sedangkan Controller menangani request, validasi, dan response, lalu Model menangani data paket foto. Contohnya, Route menggunakan `router.get("/", photoPackageController.getAll)`, Controller menggunakan `const data = getAllPhotoPackages()`, dan Model menggunakan `const getAllPhotoPackages = () => { return photoPackages; }`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan error lebih rapi karena setiap error tidak perlu menulis `res.status(404)` berulang kali pada setiap route. Pada pengujian N04, ketika ID 99 tidak ditemukan, Controller meneruskan error ke errorHandler, kemudian middleware tersebut mengirimkan response JSON dengan status 404.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	✓
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	✓
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	✓
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	✓

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100